

知ってた？

JavaScriptの " 正しさ " を検証する
テストが 5万以上もあること

(Test262)

西 悠太 / 株式会社ダイニー

自己紹介

西 悠太 (Nishi Yuta)

株式会社ダイニー / Platform Engineer

TypeScriptが大好きです。
V8も大好きです。

@riya-amemiya



今日のゴール

JavaScript が
「どのブラウザでも同じように動く」
その"当たり前"を、
誰が守っているのか？

...を知って、へー！と思って帰ってもらおう。

01

JavaScriptの仕様は
誰が決めているのか

ECMAScript と TC39

私たちが書いている JavaScript の言語仕様は **ECMAScript**。
これを制定しているのは **Ecma International** という標準化団体。

その中の **TC39** (Technical Committee 39) が ECMAScript の仕様策定を担当。

Google ・ Apple ・ Mozilla ・ Microsoft といったブラウザベンダーに加え、
Bloomberg ・ Igalia などの企業、招待された個人の専門家も参加。
特定企業ではなく、複数の利害関係者の"合議"で決まる。

早速、質問です 🙋

```
console.log("Hello, World!");
```

Q. これ、皆さんのブラウザで実行したらどうなりますか？

```
console.log("Hello, World!");
```

Chrome Hello, World!

Safari Hello, World!

Firefox Hello, World!

じゃあ、これは？

```
const hello = "Hello";  
const world = "World";  
console.log(`${hello}, ${world}!`);
```

Chrome Hello, World!

Safari Hello, World!

Firefox Hello, World!

アロー関数でも

```
const log = () => console.log("Hello, World!");  
log();
```

Chrome Hello, World!

Safari Hello, World!

Firefox Hello, World!

全部のブラウザで
同じ結果になる。
— これは偶然？

エンジンはそれぞれ別物

同じ ECMAScript を実装していても、中身は完全に独立したプロダクト。

Browser	Engine	開発
Chrome / Edge	V8	Google
Safari	JavaScriptCore	Apple
Firefox	SpiderMonkey	Mozilla
Ladybird	LibJS	Ladybird project

別チーム・別コードベース。なのに同じ結果を返す。

偶然じゃなくて、
すべてのエンジンが参照する
共通の"物差し"がある。

その物差しが

Test262

TC39 が保守する ECMAScript 公式コンFORMANCEテストスイート。

規模感

50,000+

個別のテストファイル

As of May 2025 · tc39/test262

主要エンジンはすべてこのテストを CI に組み込み、
日々の開発で仕様からの退行を検出している。

02

Test262 の中身を
見てみる

Test262 が対象とする仕様

ECMA-414 Standards Suite にまとめられた 3つの仕様。

ECMA-262 言語コア（構文・型・組み込みオブジェクト）

ECMA-402 Intl オブジェクトなどの国際化 API

ECMA-404 JSON のデータ交換フォーマット

DOM や fetch などの Web API は Test262 の対象外。
こちらは WPT（Web Platform Tests）がカバー。

少しだけ、歴史の話

競合していたベンダーが テストを持ち寄って始まった

2011年、ES5.1 リリースと同時に Test262 公開。

Test262 の源流

Google

V8 の仕様準拠検証用に開発していた
Sputnik テストスイート（5,000件超）を提供。

Microsoft

自社の ES5Conform テストスイートと、
追加テスト群を提供。

熾烈に競合していた2社が自社のテスト資産を持ち寄り、**共通の評価基準**を作った。
— Web 標準化の歴史における、競争から協調への転換点。

テストファイルの構造

各テストは **メタデータ** と **テスト本体** の2パート構成。

```
/*---  
esid: sec-array.prototype.includes  
description: Array.prototype.includes should find the target value  
features: [Array.prototype.includes]  
---*/  
  
var arr = [1, 2, 3];  
  
assert.sameValue(arr.includes(2), true);  
assert.sameValue(arr.includes(4), false);
```

— 上部のコメントブロックがメタデータ。ファイル1つにつき、仕様挙動1つが原則。

メタデータの主要キー

esid

仕様書のどのセクションを検証しているかの識別子。

例: `sec-array.prototype.includes` — 仕様書のアルゴリズムと 1:1 対応。

features

テストの実行に必要な ECMAScript 機能の宣言。

未実装機能に依存するテストを、エンジン側がスキップできる。

flags

実行モードの制御（`onlyStrict` / `noStrict` など）。

指定なしなら Strict・非 Strict 両方で実行される。

description

人間向けの短い説明。

アサーション

Test262 は独自の小さなアサーションライブラリを提供。

```
// 値の厳密な等価性
assert.sameValue(actual, expected);

// 特定の例外型のスローを検証（種類まで厳密にチェック）
assert.throws(TypeError, () => {
  null.property; // TypeError でなければ失敗
});
```

「エラーが投げられれば OK」ではない。

仕様が `TypeError` を要求する箇所で `ReferenceError` が出たら、それはテスト失敗。

新機能が仕様に入るには テストが必要

TC39 のステージプロセスでは、**Stage 2.7** でテスト執筆が本格化し、**Stage 4** への進行条件に「**2つの互換な実装が Test262 acceptance tests を通過していること**」が含まれる。



テストは 提案者・実装者・コミュニティの誰でも書ける。

Stage 2.7 — テストで設計を検証する

Stage 2.7 は 2024 年に TC39 プロセスに正式導入された比較的新しいステージ。

「設計は原則承認された。あとはテスト・実装・利用からのフィードバックで詰めるのみ」という宣言。

入る条件	完全な仕様テキスト／指定レビュアーと editor group の sign-off が揃った状態。
ここでやること	包括的な Test262 テストの作成と、実装可能性を検証する仕様準拠プロトタイプの開発。
Stage 3 との違い	Stage 3 は実装経験を積む段階。2.7 はその前提として、実装者が頼りにできるテストを用意する段階。

テストを書く → 実装者がそれを頼りに実装する → 2つの実装が通れば Stage 4

Test262 は
事後的なテストツールではなく、
言語仕様の策定プロセス
そのものに組み込まれている。

03

各エンジンは
どう動かしているか

各エンジンが Test262 を実行する方法

すべての主要エンジンが CI に統合。ローカルでも実行できる。

```
$ ./tools/run-tests.py --outdir=out/arm64.release \
  test262/language/expressions/call/eval-spread
```

```
>>> Running tests for arm64.release
```

```
>>> Running with test processors
```

```
[00:25|% 0|+ 2|- 0]: Done
```

```
>>> 56610 base tests produced 2 (0%) non-filtered tests
```

```
>>> 2 tests ran
```

test262.status — 既知の失敗リスト

実装が仕様に追いついていない箇所は、対応する Test262 テストが落ちる。

→ 失敗が分かっているテストは明示的に登録して CI を通す。

```
# https://bugs.chromium.org/p/v8/issues/detail?id=5690
```

```
'language/expressions/call/eval-spread': [FAIL],
```

test262.status は、エンジンの技術的負債の一覧表。

将来修正されるべき仕様との乖離が、ここに記録されている。

test262.fyi — 日次で公開される準拠状況

各エンジンの Test262 準拠状況は <https://test262.fyi> で毎日公開。

ナイトリービルドに対してテストを走らせ、機能サポートの差分を可視化。

- V8 / SpiderMonkey / JavaScriptCore の相互比較
- Ladybird の LibJS のような新興エンジンも掲載
- 準拠度をプロジェクトの進捗指標として活用

可視化 → エンジン間の健全な競争 → Web 全体の相互運用性の底上げ。

04

おまけ：
V8 で見つけたバグの話

直接 eval と 間接 eval

eval は呼び出し方で挙動が変わる、という前提の話。

直接 eval — ローカルスコープが見える

```
function f() {  
  const x = 10;  
  eval("console.log(x)");  
  // → 10  
}
```

間接 eval — グローバルスコープで実行

```
function g() {  
  const x = 10;  
  const e = eval;  
  e("console.log(x)");  
  // → ReferenceError  
}
```

ECMAScript 仕様で明確に定義されている挙動。エンジンは正しく判定する義務がある。

eval(...args) が壊れていた

V8 では、末尾にスプレッドがある eval 呼び出しだけ、直接 eval として認識されていなかった。

```
const args = ["1 + 2"];  
eval(...args);  
// 仕様上は直接 eval のはず  
// → V8 は間接 eval として実行していた
```

ソースコードには FIXME が残っていた

v8/src/interpreter/bytecode-generator.cc

```
// FIXME(v8:5690): Support final spreads for eval.
```

そして先ほどの `test262.status` には、同じバグ番号でテストが FAIL として登録されていた。

```
# https://bugs.chromium.org/p/v8/issues/detail?id=5690  
'language/expressions/call/eval-spread': [FAIL],
```

— FIXME と test262.status の FAIL。技術的負債の両側からの記録。

Test262 が果たした役割

この話で大事ななのは、V8 の内部実装ではなく Test262 の役割です。

- 仕様と実装のギャップを "テストの失敗" という形で可視化していた
- `test262.status` に FAIL があったから、どの仕様に違反しているかが明確だった
- 修正後も、Test262 テストの通過が 仕様準拠の証明になった

発見 → 修正 → Test262 で証明

このサイクルが「どのブラウザでも同じ結果になる」を支えている。

05

まとめ

JavaScript の"当たり前"を支える仕組み

- TC39 の合議による仕様策定
- 5万件以上のテストを日々保守するコミュニティ
- 仕様準拠を目指すエンジン開発者たち

Test262 は単なるテストツールではない。

競合するベンダーが同一の仕様を正しく実装するための、
合意形成の基盤

Test262 は、 誰にでも開かれている。

テスト追加も、エンジンの FIXME 修正も、特別な権限はいりません。
Proposal にテストを書く。コードリーディングでギャップを見つける。
それは Web の未来の"当たり前"を作る直接的な貢献です。

ご清聴ありがとうございました



スライド: github.com/riya-amemiya/amemiya_riya_slide_data

SNS等: riya-amemiya-links.tokidux.com

このスライドは CC BY-SA 4.0 でライセンスされています。
より自由な翻訳を可能にするため、翻訳は例外的に CC BY 4.0 での配布が許可されています。
Required Attribution: Riya Amemiya (<https://github.com/riya-amemiya>)